

ATmega32A Module

GPIO (Digital Input / Output)

Ali Sahafi

1 Theory

GPIO (*General Purpose Input/Output*) is the most basic peripheral: each pin of the microcontroller can be a digital **output** (drive 0 V or 5 V) or a digital **input** (read whether the outside world applies 0 V or 5 V). The ATmega32A has four 8-bit ports: **PORTA**, **PORTB**, **PORTC** and **PORTD** — 32 pins in total.

Each port is controlled by three registers ($x = A, B, C$ or D):

Register	Role	Bit meaning
DDRx	Data Direction	1 = output, 0 = input
PORTx	Output latch / pull-up	output: pin level; input: 1 = pull-up on
PINx	Input read	the actual voltage currently on the pin

Internal pull-up. An unconnected input pin *floats*: it reads random values. For buttons, enable the internal pull-up (`INPUT_PULLUP`): the pin then rests at **HIGH**, and the button connects it to GND, so **pressed = LOW**. This “active-low” logic is the standard way to wire buttons.

2 API reference

The driver is beginner-friendly and auto-corrects register names: you can pass `PORTx`, `DDRx`, or `PINx` interchangeably to any function (`setDirection`, `write`, `read`, `toggle`). The driver automatically maps to the correct hardware register under the hood, skipping register confusion.

Method	Description
<code>GPIO.setDirection(PORTx, pin, OUTPUT)</code>	Set single pin (INPUT, OUTPUT, INPUT_PULLUP)
<code>GPIO.setDirection(PORTx, ALL, INPUT_PULLUP)</code>	Set whole port direction
<code>GPIO.write(PORTx, pin, HIGH/LOW)</code>	Write single pin
<code>GPIO.write(PORTx, ALL, 0xF0)</code>	Write raw byte to whole port
<code>GPIO.write(PORTx, 0xF0)</code>	Shorthand whole-port write
<code>GPIO.read(PORTx, pin)</code>	Read single pin — returns HIGH or LOW
<code>GPIO.read(PORTx, ALL)</code>	Read whole port — returns 0–255
<code>GPIO.read(PORTx)</code>	Shorthand whole-port read
<code>GPIO.toggle(PORTx, pin)</code>	Toggle single pin
<code>GPIO.toggle(PORTx, ALL)</code>	Toggle whole port

Constants: `INPUT`, `OUTPUT`, `INPUT_PULLUP`, `HIGH`, `LOW`, `ALL` (see Module 0).

3 Example: Button-controlled LED

Board Hardware. All components are already integrated on the development board:

- **8 LEDs (D0–D7) on PORTB (PB0–PB7):** Wired active-low (cathodes to MCU pins). Driving a pin LOW turns the LED ON; driving it HIGH turns it OFF.
- **8 DIP Switches (S0–S7) on PORTC (PC0–PC7):** Connected directly to GND when closed. There are **no external pull-up resistors**, so INPUT_PULLUP must be enabled in software.
- **2 Push Buttons on PD2 and PD6:** Button S11 is connected to **PD2** and button S10 is connected to **PD6**. Pressing either button connects the pin to GND (LOW).

Build & flash. `make gpio`

```
/*
 * GPIO Example -- Button-controlled LED on ATmega32A
 * Build & flash: make gpio
 */

#define F_CPU 8000000UL
#include "gpio.hpp"

int main() {
    GPIO.setDirection(PORTD, PD2, INPUT_PULLUP); // Push button S11
    GPIO.setDirection(PORTB, PB0, OUTPUT); // LED D0 (active-low)
    GPIO.setDirection(PORTB, PB1, OUTPUT); // LED D1 (blinking)

    while (true) {
        // Button S11 controls LED D0 (active-low: LOW = pressed / ON)
        if (GPIO.read(PORTD, PD2) == LOW) {
            GPIO.write(PORTB, PB0, LOW); // LED ON
        } else {
            GPIO.write(PORTB, PB0, HIGH); // LED OFF
        }

        GPIO.toggle(PORTB, PB1); // Heartbeat blink
        _delay_ms(200);
    }
}
```

4 Laboratory Tasks

Task 1: Reading Switches and Driving LEDs (20 min)

- **Task 1.1:** Configure PORTC as INPUT_PULLUP and PORTB as OUTPUT. Write a loop that continuously reads the 8 DIP switches on PORTC and displays their status directly on the 8 LEDs on PORTB.
- **Task 1.2:** Modify the configuration of PORTC to plain INPUT (disabling the internal pull-up resistors). Observe and explain what happens when a switch is left open (floating input).

Task 2: 4-Bit Binary to Seven-Segment Decoder (20 min)

Develop a function to capture a 4-bit binary input from the switches and display the corresponding decimal digit on the seven-segment display:

- **Input Capture:** Read the lower 4 bits of the DIP switches on PORTC (PC0–PC3) using bitwise masking (value & 0x0F).
- **Value Mapping & Output:** Pass the captured 4-bit value to `SevenSeg(digit)` (`common/sevenseg.hpp`). The driver converts the digit into the correct segment pattern for the active-low common-anode display and writes it directly to PORTB.
- **Example:** If the switches are set to binary 0101, the seven-segment display should show the digit “5”.

Task 3: Bidirectional Running Light with Push-Button Control (25 min)

Apply shift operators (`<<`, `>>`), bitwise logic (`&`, `~`), and `_delay_ms()` to create an animated running light on PORTB:

- **Task 3.1 (Shift Animation):** Use a single active bit (e.g., 0x01) and shift it left and right across the 8 LEDs on PORTB (PB0 ↔ PB7) using shift operators (`<<` and `>>`) and a delay of `_delay_ms(100)`. Invert the pattern (`~pattern`) when writing to PORTB since the on-board LEDs are active-low.
- **Task 3.2 (Push-Button Control):** Integrate the two on-board push buttons on PORTD to control the animation:
 - Button S11 on **PD2**: Direct the running light to shift to the left.
 - Button S10 on **PD6**: Direct the running light to shift to the right.
- **Task 3.3 (Variable Speed via Switches):** Read the lower 2 bits of the PORTC DIP switches using a bitwise mask (`& 0x03`) and use an arithmetic expression to dynamically scale the delay duration (e.g., $50 \text{ ms} \times (\text{setting} + 1)$ for 50 ms, 100 ms, 150 ms, and 200 ms).